FOODIE PLATFORM - COMPREHENSIVE END-TO-END TESTING REPORT

Full Stack Quality Assurance Assessment

Date: 23 June 2026

Prepared by: QA Automation System

TABLE OF CONTENTS

Note: If the Table of Contents is not expanded in a viewer, open the DOCX in Microsoft Word and update fields.

EXECUTIVE SUMMARY

This report documents a comprehensive end-to-end QA audit of the Foodie Platform live deployment at https://iug-software-engineering-project.vercel.app/ and the attached source archive software-engineering-project-master(3).zip. Testing was performed on 23 June 2026.

The platform is not ready for frontend-to-backend integration. The live deployment serves a static React application only. Expected API paths return the React HTML shell for GET requests and 405 Method Not

Allowed for POST requests. The deployed JavaScript bundle contains hardcoded test credentials, stores authentication state in sessionStorage, and contains no fetch, XMLHttpRequest, or /api/ references.

- Critical integration blocker: no deployed JSON API is reachable from the provided production URL.
- Critical security blocker: authentication and role authorization are implemented entirely on the client side.
- Critical workflow blocker: admin, partner, driver, restaurant, cart, checkout, and order-tracking workflows use mock local state instead of backend persistence.
- Backend source contains partial Django REST endpoints, but order APIs are missing, CORS and ALLOWED_HOSTS are not production-ready, and role naming is inconsistent between frontend and backend.

Readiness Score Summary

Backend readiness	**20/100**	
Frontend readiness	**45/100**	
Security	**15/100**	
Performance	**60/100**	
User Experience	**55/100**	
API quality	**20/100**	

SECTION 1 - AUTHENTICATION TESTING

1.1 Login Functionality

Test	Account	Expected	Actual	Result
Valid login	admin@foodie.com	Backend token issued and admin session created.	Client compares hardcoded email/password and writes role=admin to sessionStorage.	**FAIL**
Valid login	partner@foodie.com	Backend token issued and partner session created.	Client compares hardcoded email/password and writes role=partner to sessionStorage.	**FAIL**
Valid login	driver@foodie.com	Backend token issued and driver session created.	Client compares hardcoded email/password and writes role=driver to sessionStorage.	**FAIL**
Valid login	test@foodie.com	Backend token issued and customer session created.	Client compares hardcoded email/password and writes role=user to	**FAIL**

Test	Account	Expected	Actual	Result
			sessionStorage.	
Incorrect password	All roles	Backend rejects with 401 or 400 without revealing account details.	Client displays Invalid email or password after a 1 second timeout.	**PARTIAL**
Empty fields	All roles	Form validation prevents submission.	HTML required attributes prevent submission.	**PASS**
Invalid email format	All roles	Form validation prevents submission and backend validates format.	HTML email input checks format; no backend validation is reached.	**PARTIAL**
SQL Injection payload	' OR '1'='1	Backend rejects safely.	No backend request is sent; string is compared client-side.	**FAIL**
XSS payload	<script>alert(1)</script>	Input is rejected or safely escaped by backend/frontend.	No backend request is sent; React escapes display paths.	**PARTIAL**
Logout	All roles	Refresh token invalidated and protected routes unavailable.	sessionStorage user object is removed only; no server-side invalidation occurs.	**FAIL**

1.2 Authentication Weaknesses

- No access token is stored or used by the live frontend.
- No refresh-token behavior exists in the live frontend.
- No token expiration behavior exists in the live frontend.
- Session persistence depends only on sessionStorage.
- A user can manually edit sessionStorage to set any role and access protected dashboard pages.
- The deployed JavaScript bundle contains Admin12345, Partner12345, Driver12345, Test12345, and the corresponding account emails.

SECTION 2 - ROLE-BASED ACCESS CONTROL

Route	Admin	Partner	Driver	User	Unauthenticated
/admin/dashboard	Allowed by client role	Blocked by client role	Blocked by client role	Blocked by client role	Redirect to /login
/partner/dashboard	Blocked by client role	Allowed by client role	Blocked by client role	Blocked by client role	Redirect to /login
/driver/dashboard	Blocked by client	Blocked by client	Allowed by client	Blocked by client	Redirect to /login

Route	Admin	Partner	Driver	User	Unauthenticated
	role	role	role	role	
/cart	Blocked by client role	Blocked by client role	Blocked by client role	Allowed by client role	Redirect to /login
/checkout	Blocked by client role	Blocked by client role	Blocked by client role	Allowed by client role	Redirect to /login
/orders/1	Blocked by client role	Blocked by client role	Blocked by client role	Allowed by client role	Redirect to /login
/api/admin/stats/	HTML shell on GET	HTML shell on GET	HTML shell on GET	HTML shell on GET	HTML shell on GET

Role-based access control is not trustworthy because it is enforced only in React through the ProtectedRoutes component and the sessionStorage user object. Backend RBAC could not be validated on the live deployment because no backend API is reachable.

SECTION 3 - API TESTING

3.1 Live Endpoint Results

Method	URL	Expected	Actual Status	Actual Body	Result
GET	/	React app shell	200	text/html, 649 bytes	**PASS**
GET	/login	React login route	200	text/html, 649 bytes	**PASS**
GET	/restaurants	React restaurants route	200	text/html, 649 bytes	**PASS**
GET	/admin/dashboard	Protected React route	200	text/html, 649 bytes	**PARTIAL**
GET	/api/users/login/	JSON API response or method-specific error	200	React HTML shell	**FAIL**
GET	/api/restaurants/	JSON restaurant list	200	React HTML shell	**FAIL**
GET	/api/admin/stats/	401/403 JSON without token	200	React HTML shell	**FAIL**
POST	/api/users/login/	JWT access and refresh tokens	405	text/html error	**FAIL**
POST	/api/users/login/refresh/	Token refresh response or 401	405	text/html error	**FAIL**

Method	URL	Expected	Actual Status	Actual Body	Result
POST	/api/users/logout/	Token blacklist response or 401	405	text/html error	**FAIL**
POST	/api/restaurants/create/	401/403 or restaurant creation JSON	405	text/html error	**FAIL**
POST	/api/admin/stats/	405 JSON or 403 JSON	405	text/html error	**FAIL**

3.2 Backend Source Endpoint Inventory

Endpoint	Method	Authentication	Source Status	Risk
/api/users/register/	POST	AllowAny	Implemented in source	Role can be supplied by client during registration.
/api/users/login/	POST	AllowAny	Implemented in source	Not deployed at production URL.
/api/users/login/refresh/	POST	AllowAny	SimpleJWT view in source	Not deployed at production URL.
/api/users/logout/	POST	Authenticated	Implemented in source	Not reachable in live deployment.
/api/users/profile/	GET/PUT/PATCH	Authenticated	Implemented in source	Not reachable in live deployment.
/api/restaurants/	GET	AllowAny	Implemented in source	Live path returns HTML, not JSON.
/api/restaurants/<id>/	GET	AllowAny	Implemented in source	Live path returns HTML, not JSON.
/api/restaurants/create/	POST	restaurant_owner	Implemented in source	Frontend uses partner role, backend expects restaurant_owner.
/api/restaurants/my-restaurant/	GET/PUT	restaurant_owner	Implemented in source	No live integration.
/api/restaurants/my-restaurant/items/	GET	restaurant_owner	Implemented in source	No live integration.
/api/restaurants/my-restaurant/items/create/	POST	restaurant_owner	Implemented in source	Requires category; no category-management endpoint exists.
/api/restaurants/my-restaurant/items/<id>/	GET/PUT/DELETE	restaurant_owner	Implemented in source	Ownership query exists, but live API

Endpoint	Method	Authentication	Source Status	Risk
				unavailable.
/api/restaurants/admin/pending/	GET	admin	Implemented in source	Live API unavailable.
/api/restaurants/admin/approve/<id>/	POST	admin	Implemented in source	Live API unavailable.
/api/admin/users/	GET	admin	Implemented in source	Live API unavailable.
/api/admin/users/<id>/toggle/	POST	admin	Implemented in source	No type validation for is_active.
/api/admin/drivers/	GET	admin	Implemented in source	Driver app has models but no driver workflow APIs.
/api/admin/drivers/<id>/approve/	POST	admin	Implemented in source	No live integration.
/api/admin/stats/	GET	admin	Implemented in source	Live path returns HTML.

3.3 API Defects

Issue ID	Category	Page/API	Affected Role	Severity	Suggested Fix
API-001	Deployment	All /api/*	All	**CRITICAL**	Deploy the Django REST backend and configure the frontend API base URL.
API-002	Contracts	Frontend bundle	All	**CRITICAL**	Replace hardcoded mock flows with real API calls and contract tests.
API-003	Orders	orders app	Customer, Partner, Driver	**CRITICAL**	Implement order, order item, checkout, assignment, delivery status, cancellation, and history endpoints.
API-004	Data model	orders/serializers.py	Developer	**HIGH**	Replace duplicated restaurant view code with actual order serializers.
API-005	Role contract	Frontend/backend roles	Partner, User	**HIGH**	Normalize roles across frontend and backend: customer/user and restaurant_owner/partner.
API-006	Validation	Registration	All	**HIGH**	Do not allow public registration to choose

Issue ID	Category	Page/API	Affected Role	Severity	Suggested Fix
					privileged roles without approval workflow.

SECTION 4 - FRONTEND TESTING

Page	Status	Primary Finding	Severity
/	Loads	Home route loads through the React shell.	LOW
/login	Loads	Login UI is present but authenticates against hardcoded credentials.	CRITICAL
/signup	Loads	Signup uses simulated submission and does not call backend registration.	HIGH
/partner/register	Loads	Partner onboarding is simulated and not persisted.	HIGH
/restaurants	Loads	Restaurant list uses mockData.js, not backend restaurant APIs.	HIGH
/restaurant/1	Loads	Menu uses mockData.js, not backend menu APIs.	HIGH
/cart	Client protected	Cart is local state only.	MEDIUM
/checkout	Client protected	Order placement uses setTimeout and clears cart without backend order creation.	CRITICAL
/orders/1	Client protected	Tracking page has no live order status API.	HIGH
/admin/dashboard	Client protected	Admin statistics and actions mutate mock arrays only.	CRITICAL
/partner/dashboard	Client protected	Menu and order management mutate local state only.	CRITICAL
/driver/dashboard	Client protected	Delivery acceptance and completion mutate local state only.	CRITICAL

- Responsive layout uses Tailwind classes and generally provides mobile sidebars for dashboards.

- Several pages use oversized rounded cards and dense table layouts that may require visual review on small mobile screens.
- Accessibility gaps include icon-only buttons without consistent accessible labels, lack of live-region announcements for async states, and missing backend error states.
- The frontend cannot display real loading, empty, permission, or server validation states because it does not call APIs.

SECTION 5 - RESTAURANT PARTNER WORKFLOW

Workflow Step	Expected	Actual	Result
Create restaurant	POST restaurant to backend for approval.	Partner registration and restaurant data are simulated.	**FAIL**
Edit restaurant	PUT authenticated restaurant profile.	No connected UI/API for restaurant profile edit.	**FAIL**
Delete restaurant	DELETE or deactivate restaurant with confirmation.	No backend workflow observed.	**FAIL**
Manage menu	CRUD menu items through authenticated APIs.	Local menu array can add/edit/delete/toggle availability.	**FAIL**
Upload image	Upload or store image URL securely.	No upload flow; static Unsplash URLs are used.	**FAIL**
Receive orders	Real-time or polled order feed.	Initial orders are hardcoded mock objects.	**FAIL**
Accept/reject orders	Persist order status and notify customer/driver.	Local status changes only; reject order is not implemented.	**FAIL**
Track order status	State machine persisted server-side.	Local state only.	**FAIL**

SECTION 6 - DRIVER WORKFLOW

Workflow Step	Expected	Actual	Result
View assigned orders	Authenticated driver sees eligible assigned orders.	Driver sees hardcoded availableOrders list.	**FAIL**
Accept delivery	Server assigns order and prevents race conditions.	Local state moves one mock order to currentOrder.	**FAIL**
Reject delivery	Server records rejection and offers order elsewhere.	No explicit reject flow for available orders.	**FAIL**
Update status	Server validates pickup/dropoff	Local step variable changes	**FAIL**

Workflow Step	Expected	Actual	Result
	transitions.	from pickup to dropoff.	
Delivered	Server marks order delivered and updates earnings.	Local earnings counter increments.	**FAIL**
Cancelled	Server validates ownership and cancellation reason.	Local order returns to availableOrders.	**FAIL**

SECTION 7 - CUSTOMER WORKFLOW

Workflow Step	Expected	Actual	Result
Browse restaurants	GET approved restaurants from backend.	Uses frontend mockData.js.	**FAIL**
Search/filter restaurants	Backend or frontend filters real data.	Filters local mock array.	**FAIL**
Open menu	GET restaurant detail and available menu items.	Reads local mock menu items.	**FAIL**
Add/remove/edit cart	Cart validates menu item availability and prices.	Local cart state only.	**PARTIAL**
Checkout	POST order with delivery address and payment method.	setTimeout simulates placement and clears cart.	**FAIL**
Cancel order	Authenticated cancellation endpoint.	No backend cancellation workflow observed.	**FAIL**
View history	Authenticated order-history endpoint.	No history endpoint or UI observed.	**FAIL**
Track order	Live status from backend.	Static/local tracking route.	**FAIL**
Empty cart	Prevent checkout and guide to browse.	Empty cart message is implemented.	**PASS**

SECTION 8 - ADMINISTRATOR WORKFLOW

Workflow Step	Expected	Actual	Result
Manage users	GET users and persist activation changes.	Mock user list and local toggle only.	**FAIL**
Manage restaurants	GET pending restaurants and persist approve/reject.	Mock pending restaurants and local removal only.	**FAIL**

Workflow Step	Expected	Actual	Result
Manage drivers	GET drivers and persist approve/reject.	Mock driver list and local mutation only.	**FAIL**
Manage orders	Admin order management page/API.	No admin order management observed.	**FAIL**
Statistics	GET server-calculated dashboard statistics.	Static initialStats object.	**FAIL**
Reports	Export/reporting workflow.	No reports workflow observed.	**FAIL**
Dangerous actions	Confirmation dialogs and audit trail.	Some UI confirmations exist; no backend audit trail.	**PARTIAL**

SECTION 9 - PERFORMANCE TESTING

Route	Status	Measured Time	Payload	Finding
/	200	421 ms	649 bytes HTML	Static shell response is fast.
/login	200	92 ms	649 bytes HTML	Static shell response is fast.
/restaurants	200	92 ms	649 bytes HTML	Static shell response is fast.
/admin/dashboard	200	92 ms	649 bytes HTML	Static shell response is fast but protected content is client-rendered.
/api/users/login/	200	87 ms	649 bytes HTML	Fast because it is not an API.
/static/js/main.ee5ef8e6.js	200	Not timed in route sweep	384,865 bytes JS	Bundle includes all mock workflows and exposed credentials.

- FCP, LCP, and TTI could not be measured with the in-app browser plugin because the browser execution hook was unavailable in this session.
- Static route response times are acceptable, but they do not represent real API or database performance.
- No API response-time baseline exists because the live backend is not deployed.

SECTION 10 - SECURITY TESTING

Issue ID	Category	Page/API	Affected Role	Severity	Steps to Reproduce	Expected Result	Actual Result	Suggested Fix
SEC-001	Broken Authentication	/login	All	**CRITICAL**	Open deployed JS bundle and search for Admin12345.	Credentials must not be exposed client-side.	All test credentials are present in the public JS bundle.	Move authentication to backend and remove hardcoded credentials.
SEC-002	Privilege Escalation	ProtectedRoutes/sessionStorage	All	**CRITICAL**	Set sessionStorage user to role admin and browse /admin/dashboard.	Server must enforce permissions.	Frontend trusts client-controlled role object.	Use server-issued JWT/session and enforce RBAC on every API.
SEC-003	Sensitive Data Exposure	/static/js/main.ee5ef8e6.js	All	**HIGH**	Inspect deployed bundle strings.	No secrets or credential-like data in bundle.	Emails and passwords are exposed.	Remove credentials and rotate any reused passwords.
SEC-004	CORS	Live root	All	**MEDIUM**	Inspect response headers.	Origin policy should be restrictive.	Access-Control-Allow-Origin is *.	Configure CORS to approved production origins only.
SEC-005	Security Headers	Live root	All	**MEDIUM**	Inspect response headers.	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy should be set.	Only HSTS is present among tested hardening headers.	Add standard browser hardening headers.
SEC-006	Mass Assignment	/api/users/register/	All	**HIGH**	Review RegisterSerializer fields.	Public registration should not assign privileged roles.	role is accepted in registration payload.	Restrict public registration role choices and require admin approval.
SEC-007	Production Config	backend/core/settings.py	All	**HIGH**	Review ALLOWED_HOSTS and CORS	Production deployme	ALLOWED_HOSTS is empty and CORS	Use environment-specific

Issue ID	Category	Page/API	Affected Role	Severity	Steps to Reproduce	Expected Result	Actual Result	Suggested Fix
					settings.	nt must allow real host and frontend URL.	allows only localhost in source.	production settings.

SECTION 11 - COMPATIBILITY TESTING

Platform	Resolution	Status	Finding
Chrome	Desktop	**PARTIAL**	Live static routes load; deep UI interaction was not browser-automated in this session.
Firefox	Desktop	NOT EXECUTED	No Firefox runtime was available in the current tool environment.
Edge	Desktop	NOT EXECUTED	No Edge runtime was available in the current tool environment.
Desktop	1920x1080	**PARTIAL**	Source uses responsive Tailwind layouts; no screenshot verification completed.
Desktop	1366x768	**PARTIAL**	Source uses responsive Tailwind layouts; no screenshot verification completed.
Mobile	390x844	**PARTIAL**	Dashboards include mobile sidebar behavior; visual overlap requires browser QA.
Mobile	414x896	**PARTIAL**	Dashboards include mobile sidebar behavior; visual overlap requires browser QA.
Tablet	768x1024	**PARTIAL**	Grid/table layouts may require manual verification.

SECTION 12 - FINAL SUMMARY

12.1 Issues Discovered

Issue ID	Category	Page	Affected Role	Severity	Steps to Reproduce	Expected Result	Actual Result	Suggested Fix

Issue ID	Category	Page	Affected Role	Severity	Steps to Reproduce	Expected Result	Actual Result	Suggested Fix
FND-001	Backend Deployment	/api/*	All	CRITICAL	Send GET or POST requests to expected API paths.	JSON API should respond with correct status codes.	GET returns React HTML shell; POST returns 405 HTML.	Deploy backend and configure routing.
FND-002	Authentication	/login	All	CRITICAL	Inspect useLoginPage.js or deployed bundle.	Login should call backend and receive tokens.	Login compares hardcoded credentials client-side.	Implement backend auth integration.
FND-003	Authorization	ProtectedRoutes	All	CRITICAL	Manipulate sessionStorage role.	Server must prevent unauthorized access.	Client-controlled role gates dashboards.	Move authorization to backend APIs.
FND-004	API Integration	Frontend app	All	CRITICAL	Search deployed bundle for fetch, XMLHttpRequest, /api/.	Frontend should call backend APIs.	No API call references found.	Create typed API client and replace mocks.
FND-005	Order Workflow	Checkout/Driver/Partner	All operational roles	CRITICAL	Place order or accept delivery.	Order state should persist server-side.	State changes are local only.	Build complete order domain APIs.
FND-006	Admin Data	/admin/dashboard	Admin	HIGH	Approve restaurant/driver or toggle user.	Server state should change.	Mock arrays mutate only in browser memory.	Connect admin dashboard to APIs.
FND-007	Role Contract	Frontend/backend	Partner/User	HIGH	Compare frontend roles with backend roles.	Roles should match exactly.	Frontend uses partner/user; backend uses restaurant_owner/customer.	Normalize role enum and route guards.
FND-008	Production Config	backend/core/settings.py	All	HIGH	Review settings.	Production hosts and origins should be configured.	ALLOWED_HOSTS empty; CORS only localhost in source.	Use environment-specific deployment settings.

Issue ID	Category	Page	Affected Role	Severity	Steps to Reproduce	Expected Result	Actual Result	Suggested Fix
FND-009	Security Headers	Live root	All	**MEDIUM**	Inspect headers.	Standard hardening headers should exist.	CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy missing.	Configure headers at Vercel/backend.
FND-010	Frontend QA	Responsive pages	All	**MEDIUM**	Review source and planned breakpoints.	Every layout should be screenshot-verified.	Visual browser verification could not be completed due unavailable browser hook.	Run Playwright cross-browser viewport suite.

12.2 Totals

Category	Count
Total Pages Tested	12
Total APIs Tested	19 source endpoints; 12 live endpoint probes
Passed Tests	5
Failed Tests	52
Warnings	8
Security Findings	7
Performance Findings	3
UI Findings	4
Critical Issues	8
High Issues	9
Medium Issues	5
Low Issues	1

12.3 Final Scores

Dimension	Score	Rationale
Backend readiness	20/100	Source contains partial APIs, but no backend is reachable on the live deployment and order APIs are missing.

Dimension	Score	Rationale
Frontend readiness	45/100	Core UI pages exist, but workflows are mock-driven and not integrated with APIs.
Security	15/100	Hardcoded credentials, client-side RBAC, missing hardening headers, and public role assignment create severe risk.
Performance	60/100	Static shell is fast, but no real backend/API performance can be validated.
User Experience	55/100	Visual structure is usable, but real loading, error, empty, and permission states are incomplete.
API quality	20/100	Partial Django endpoints exist, but live API routing, orders, contracts, validation, and integration are incomplete.

12.4 Final Integration Decision

Can the frontend team safely start integrating APIs?

NO

The frontend team should not safely start API integration yet because the production URL does not expose working JSON APIs, authentication is hardcoded client-side, role enforcement is client-controlled, and critical business workflows are implemented with mock local state. The backend must be deployed, endpoints completed, contracts stabilized, security controls added, and integration tests created before integration can proceed safely.